

An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Acceleration for VLLM Inference

Liang Chen¹, Haozhe Zhao¹, Tianyu Liu^{2†}, Shuang Bai², Junyang Lin²

Chang Zhou², Baobao Chang^{1†}

¹National Key Laboratory for Multimedia Information Processing, Peking University

²Alibaba Group

leo.liang.chen@outlook.com

Abstract

In this study, we identify the inefficient attention phenomena in Large Vision-Language Models (LVLMs), notably within prominent models like LLaVA-1.5, QwenVL-Chat, and Video-LLaVA. We find that the attention computation over visual tokens is extremely inefficient in the deep layers of popular LVLMs, suggesting a need for a sparser approach compared to textual data handling. To this end, we introduce FastV, a versatile plug-and-play method designed to optimize computational efficiency by learning adaptive attention patterns in early layers and pruning visual tokens in subsequent ones. Our evaluations demonstrate FastV’s ability to dramatically reduce computational costs (e.g., a 45% reduction in FLOPs for LLaVA-1.5-13B) without sacrificing performance in a wide range of image and video understanding tasks. The computational efficiency and performance trade-off of FastV are highly customizable and Pareto-efficient. It can compress the FLOPs of a 13B-parameter model to achieve a lower cost than that of a 7B-parameter model while still maintaining superior performance. We believe FastV has practical value for the deployment of LVLMs in edge devices and commercial models. Code is released at github.com/pkunlp-icler/FastV.



Figure 1: The Efficiency/Performance trade-off curve of FastV. The x-axis stands for the theoretical FLOPs reduction ratio under different FastV configurations. The y-axis stands for performance under different settings, we report the average scores of {Nocaps (Cider), Flickr30k (Cider), A-OKVQA (Acc), MMMU (Acc)}. We can see that FastV can achieve 45% FLOPs reduction with nearly no performance loss for different models.

[†]Corresponding author.

1 Introduction

Large Vision-Language Models (LVLMs) have become a hit in both computer vision and natural language processing studies. We have witnessed tremendous creative research and applications that are built upon powerful LVLMs Liu et al. (2023c; 2024a); Team et al. (2023); Bai et al. (2023). From describing the given picture to navigating the internet Zheng et al. (2024), using smartphones Wang et al. (2024) and making decisions in the real world Driess et al. (2023); Chen et al. (2024), large language models with vision abilities are reshaping how we interact with AI systems, which cannot be achieved solely by language or vision uni-modal models.

Currently, a majority of popular LVLMs rely on sequential visual representation, where images are transformed into hundreds or thousands of tokens when feeding them to LLM along with language prompts OpenAI (2023); Zhu et al. (2023); Liu et al. (2023c); Zhao et al. (2023); Bai et al. (2023). As LVLMs leverage the advanced emergent capabilities inherent in their language components, they concurrently face a surge in computational complexity, correlating with cost increments. This complexity stems from the principle that the proficiency of Large Language Models (LLMs) is predominantly influenced by their scale. Two critical areas remain under-explored in this context: 1) How do language models process and interpret images? and 2) While the efficient training and inference of LLMs have attracted considerable attention, these dimensions within LVLMs are yet to be thoroughly examined and understood.

In this paper, we uncover the fact that current LVLMs actually apply an inefficient way while processing image information. Specifically, the image tokens receive strikingly lower attention scores compared to their textual counterparts within the token-based LVLMs like LLaVA. The degree of imbalance also varies between the shallow and deep layers. In the image captioning tasks, we observed that within the deep layers (after layer 2) of renowned LVLMs such as LLaVA 1.5, image tokens garner an average attention score that amounts to only 0.21% of the score attributed to system prompts. In contrast, this figure reaches 50% in the initial two layers. These observations raise questions upon the optimal utilization of visual information within LVLMs.

To address the problem, we assume a plausible explanation is that the high redundancy in visual signals leads to the aggregation of image-related, instruction-specific features onto certain “anchor” tokens through the self-attention mechanism in the shallow layers. Notably, these anchor tokens are not image tokens. In deep layers, attentions are focused on those anchor tokens, leading to significantly reduced attention on the image tokens themselves.

The phenomena inspires to propose FastV, a dynamic image tokens pruning method to reduce the inference cost of LVLMs. Our findings suggest an intriguing possibility: Given that image tokens contribute minimally to output generation in deeper layers due to diminished attention, why not consider removing them at these stages? FastV implements an image token pruning strategy at one specific layer of LLM. Prior to this layer, computations proceed as usual. Beyond this selected layer, image tokens are re-evaluated based on their average received attention scores. Tokens falling below a predefined attention score threshold are then selectively discarded in subsequent layers, streamlining the process by focusing on the most impactful tokens.

Compared to other attention-based methods for accelerating inference, such as sparse attention, FastV’s most notable distinction lies in its direct elimination of tokens. This approach not only bypasses the computational demand of the self-attention module but also the Feed-Forward Network (FFN) module in deeper layers. As a result, FastV achieves a great theoretical reduction in FLOPs while maintaining relatively high performance as shown in Figure 1’s experiment on LLaVA and Qwen-VL-Chat models. Our experiment on LLaVA-1.5-13B model shows that we can filter out 50% image tokens after layer 2 without sacrificing the average performance on a combination of Vision-Language tasks including captioning tasks like Nocaps Agrawal et al. (2019), Flickr30K Plummer et al. (2015), multiple choice tasks like A-OKVQA Schwenk et al. (2022), MMMU Yue et al. (2023), complex embodied reasoning task like PCA-Bench Chen et al. (2024; 2023), tasks requiring detailed OCR abtily like OCR-VQA Mishra et al. (2019), more challenging video

understanding tasks Jang et al. (2017); Xu et al. (2017a;b) and more fine-grained evaluation like MME Fu et al. (2023), MMVet Yu et al. (2023) and SeedBench Li et al. (2023a). Our latency test experiment on A-OKVQA showed that LLaVA-13B model with FastV could achieve a lower latency than LLaVA-7B model while maintaining superior performance. This result highlights the effectiveness of FastV in balancing the trade-off between speed and accuracy in LVLMs.

Researches Liu et al. (2023c); Li et al. (2023e) underscore the significance of enhancing image resolution for the performance of LVLMs. However, it’s equally important to note that increased resolution comes with its own challenges, including a rise in the computational costs such as longer image token sequence and inference latency. We also conduct experiments on training LVLM in different image feature resolution by setting pooling layer of different strides. Specifically, with an equal number of image tokens, models equipped with FastV can process higher resolution images, leading to better performance than models limited to lower resolution features. This finding highlights the potential to enhance downstream performance by increasing image resolution without incurring additional inference costs.

In summary, the contribution of the work are three-folds:

1. Identify and analyze the inefficient visual attention phenomena in prevailing LVLMs.
2. Propose FastV, a plug-and-play method to significantly reduce inference cost for LVLMs without sacrificing performance inspired by our observation.
3. Validate the effectiveness of FastV on a wide range of vision-language tasks across different LVLMs with thorough ablations.

2 Related Work

Large Vision-Language Model. To benefit from the advancement of LLM and integrate visual information into the LLM, large Vision-Language Models utilize a Visual Prompt Generator Li et al. (2023b) to transform the visual embeddings into prompts that the language model can comprehend Li et al. (2023c); Liu et al. (2023c), resulting in a significant increase in required tokens. Handling higher resolution images inevitably necessitates a quadratic increase in the number of needed tokens. For instance, LLaVA process 336x336 images into 576 tokens Liu et al. (2023b) and process images with a greater resolution of 672x672 into 2304 tokens Liu et al. (2024b). Fuyu Bavishi et al. (2023), in a similar vein, translates pixel-level images of 1080x1080 into 1296 tokens. Understanding and generating multiple images or videos also inherently demands an escalated count of tokens for vision information. Both Video-Poet Kondratyuk et al. (2023) and Unified-IO2 Lu et al. (2023) are compelled to reserve thousands of tokens within the context to facilitate the understanding and generation of multiple images or videos. Large multimodal models like Gemini Team et al. (2023) and LWM Liu et al. (2024a) highlights the significance of long context in developing a robust understanding of the world model and extending the context length to 1M to address the issue of escalating context requirements.

Inference Optimization for LLM. Efficient inference in LLMs is challenged by their autoregressive generation where each token prediction depends on the preceding context. Hence, considering the quadratic complexity of computation’s attention during training, as the context length increases, the generation becomes progressively slower. To tackle these challenges, pioneering studies fall into two categories: methods optimizing memory consumption for attention module like FlashAttention, vLLM and RingAttention Dao et al. (2022); Dao (2023); Kwon et al. (2023); Liu et al. (2023a), which ensure no drastic shifts in the results, and methods like StreamingLLM and FastGen Xiao et al. (2023); Ge et al. (2024) that simplify computations by pruning redundant attention computation. We are interested in the second kind of methods since they are proposed inspired by the distinct attention patterns observed in LLM’s inference. While these methods have boosted the inference efficiency of LLMs, they are designed for text-only language models, and whether their effectiveness can be transferred to LVLMs remain under-explored. There is previous work

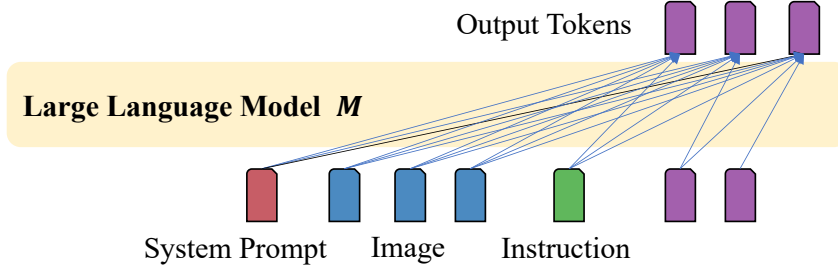


Figure 2: Classic network architecture of LVLm. Image tokens and different types of text tokens are sent to the LLM as input. LLM generates output tokens conditioned on the input tokens and preceding output in an auto-regressive manner.

attempt to handle the long-context in LVLms efficiently, like LLaMA-VID Li et al. (2023d), which utilizes cross-attention to effectively represent each video frame with two key tokens, the requirement for an additional fine-tuning stage obstructs its broad applicability for different LVLms.

Token Reduction for VLMs. There have been studies on improving efficiency for Vision-Language Models (VLMs) before the era of large vision-language models. A majority of them focus on token reduction for vision transformers (ViTs). Various methods, such as EViT Liang et al. (2022), SPViT Kong et al. (2022), and Pumer Cao et al. (2023), have been proposed for ViTs. More recently, PYRA Xiong et al. (2024) has enhanced the training and inference of ViTs via a specialized token merging technique. FastV is the first to explore visual token reduction for Large Vision-Language Models (LVLms), which uses language as an interface for various vision-language tasks. FastV utilizes the signal from LLM to guide the pruning of visual tokens, a strategy not previously explored. We are the first to demonstrate the effectiveness of token reduction in video-QA and various comprehensive LVLm benchmarks. Another significant advantage of FastV over previous methods is its simplicity; it can be applied to any LVLm without requiring model retraining.

3 Inefficient Visual Attention in VLLMs

3.1 Preliminaries

In this section, we delve into how LVLms process visual tokens during output generation from the perspective of self-attention module. For an image-question pair (d, t) , the given LVLm M , usually in the structure of transformer Vaswani et al. (2017) decoder, predicts the answer $\hat{y} = M(d, t)$ in an auto-regressive manner:

$$p(\hat{y}) = \prod_{i=1}^N p_M(\hat{y}_i | \hat{y}_{1:i-1}; d; t) \quad (1)$$

Multimodal information, encompassing both images and text, is transformed into sequential embeddings prior to being processed by the transformer model. For images, a commonly used approach is to employ a pretrained encoder, such as CLIP-ViT Radford et al. (2021), to extract visual features. These features are then linearized by eliminating the spatial dimension. Additional linear transformations Zhu et al. (2023); Liu et al. (2023b) or cross-attention Li et al. (2023c); Bai et al. (2023) modules are utilized to adjust the size of the visual features to match the embedding size of the Large Language Model (LLM) and to achieve semantic alignment. Regarding text, a tokenizer breaks down the natural language into discrete tokens and then performs an embedding lookup to form text embeddings. In the rest of the paper, we refer to ‘visual tokens’ and ‘text tokens’ not merely as the discrete units of visual and textual data but as the embeddings derived from these units.

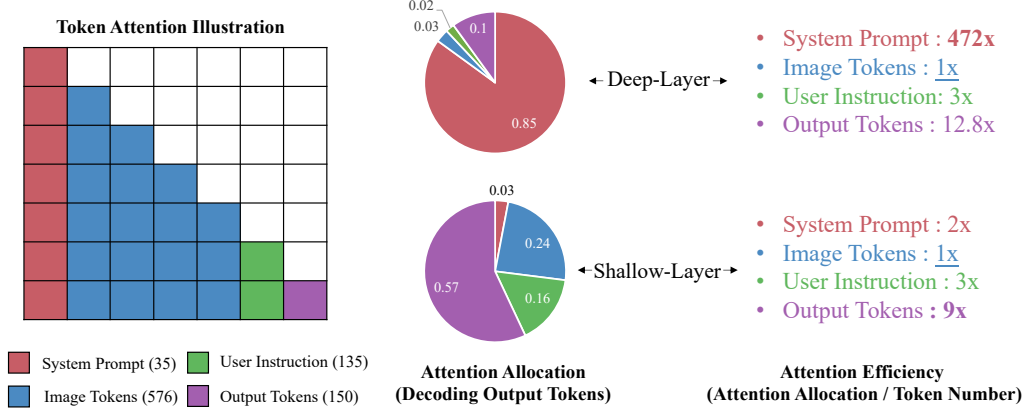


Figure 3: Illustration of inefficient visual attention phenomena. The left part shows the relative position and average number of different type of input tokens, tokens could only attend to preceding tokens in the self-attention module. In average, image tokens take up most of the input tokens (64%). The middle and right part show the average attention allocation λ and attention efficiency ϵ in shallow and deep layers. Image tokens receive far less attention relative to their number in the deep layers.

As illustrated in Figure 2, after preprocessing the image and text token to a unified embedding space, they are fed to the transformer decoder to generate output tokens. The input tokens at each decoding step can be categorized into four distinct types: system prompt (sys), image tokens (img), user instruction (ins), and output tokens (out). The system prompts for LVLMs usually inherit the backbone LLM, used as a general message to control the LLM’s behavior, which is decided during the instruction tuning stage of LLM. Image tokens are the linearized image features transformed by a pretrained vision encoder. User instruction specifies the query question for the given image. Output tokens are generated step by step conditioned on the preceding tokens.

3.2 Experiment Settings

To explore how LVLMs process image tokens, we first randomly sample N image-text pairs $D = \{(d^1, t^1), \dots, (d^N, t^N)\}$ from a combination of vision language tasks including image caption (Flickr30K), embodied reasoning (PCA-Bench), visual question answering (A-OKVQA), multimodal understanding and reasoning (MMMU) and then prompt the LVLm to generate N responses $\hat{Y} = \{\hat{y}^1, \dots, \hat{y}^N\}$.

During the decoding process of one response, we collect each output tokens’ attention score distribution α in different layers and sum up for different type of input tokens. That is, for the i -th token, in the j -th layer, we compute $\alpha_{sys}^{i,j}, \alpha_{img}^{i,j}, \alpha_{ins}^{i,j}, \alpha_{out}^{i,j}$ to denote the total attention score current token attends to the system prompt, image tokens, user instruction and output tokens. We have:

$$\alpha_{sys}^{i,j} + \alpha_{img}^{i,j} + \alpha_{ins}^{i,j} + \alpha_{out}^{i,j} = 1 \quad (2)$$

We compute the total attention allocation λ to denote the total attention score one type of tokens received in one layer. For example, the total attention of system prompt in layer j is:

$$\lambda_{sys}^j = \sum_{i=1}^n \alpha_{sys}^{i,j} \quad (3)$$

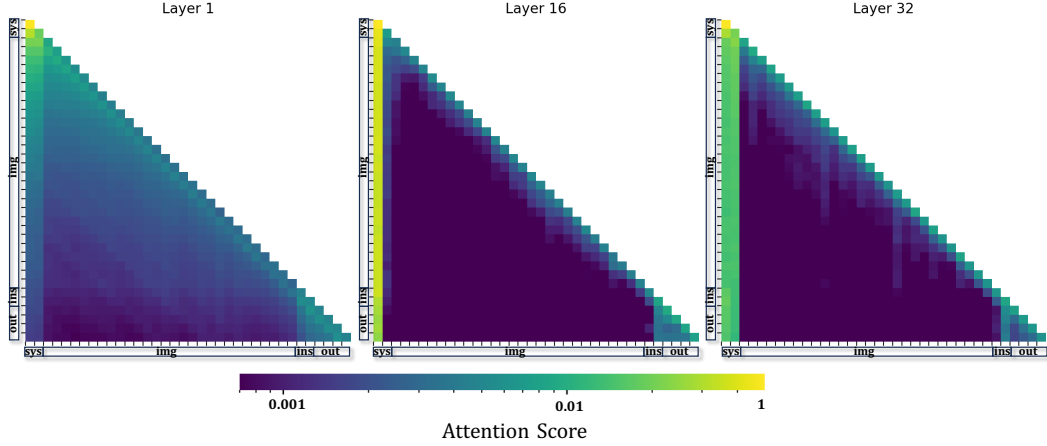


Figure 4: The attention maps during the decoding process of one model response for LLaVA1.5-7B. We can see that in the bottom layer, attention distributes relatively smooth across different type of tokens. In the the deep layers, above from local attention, the attention scores are aggregated to system prompt, instruction and output tokens and attention over image tokens is rather sparse.

where n is the number of tokens in the response. Final attention allocation is averaged over all attention heads in the N image-text pairs we sampled.

Next, we define metric **attention efficiency** ϵ to denote the average attention score per type’s token received in one layer during the decoding process of one response. For example, the attention efficiency of image tokens in layer j is:

$$\epsilon_{img}^j = \frac{\sum_{i=1}^n \alpha_{img}^{i,j}}{|img|} \quad (4)$$

where $|img|$ is the number of image tokens, n is the number of tokens in the response. Final attention efficiency is averaged over all attention heads in the N image-text pairs we sampled.

In our experiment, N is set to 1000 and we use LLaVA1.5-7B as the LVLM. We follow the same generation configuration as the original paper Liu et al. (2023c).

3.3 Results

We have two major findings in the attention pattern statistics regarding attention allocation λ and attention efficiency ϵ for different type of input tokens. We define the first 2 layers as shallow layer and the rest 30 layers as deep layers.

1. Both attention allocation and attention efficiency show different degree of imbalance, which is related to the layer depth. The average attention allocation and efficiency in different layer is shown in Figure 3. In shallow layer the attention allocation is relatively more balanced than in deep layers. In shallow layer, the output tokens tends to attend to the previous output tokens while in deep layers, they tend to attend to the system prompt.
2. Image tokens have the **lowest** attention efficiency in both shallow and deep layers. System prompt is of extremely high attention efficiency in deep layers, which is **472** times that of image tokens, taking up 85% total attention scores.

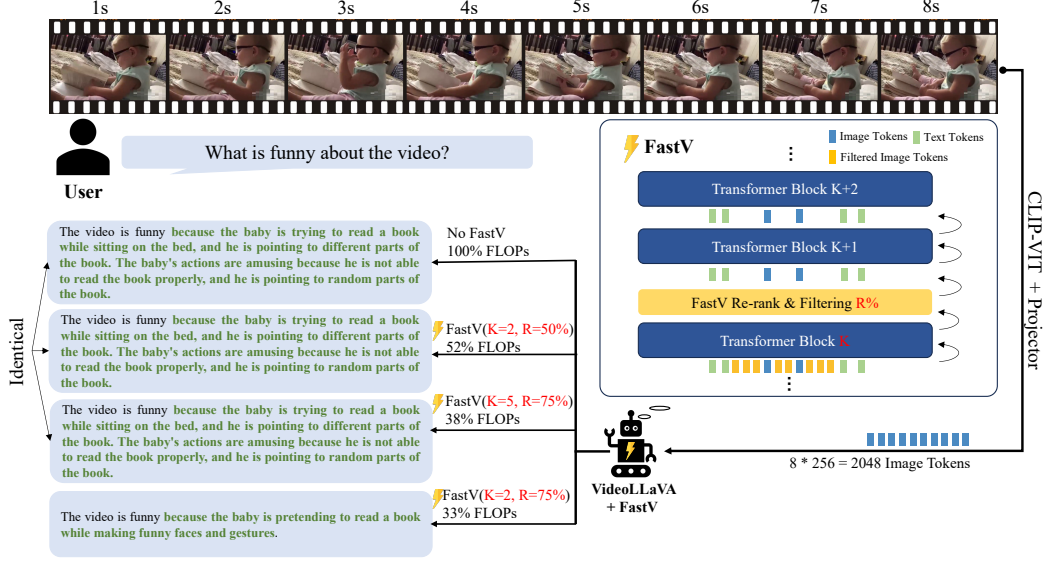


Figure 5: Illustration of FastV. For image or video input (multiple image frames), they are first transformed to visual tokens with a pretrained image encoder like CLIP-ViT and then processed by the LLM decoder. FastV dynamically prunes $R\%$ image tokens after layer K in the forward process of input tokens. We can tell from the output that FastV does not influence the correctness while reducing significant FLOPs. The correct facts in the outputs are marked **green**. The first three outputs are completely identical.

3.4 Insights

The statistics reveal a surprising trend in the decoding process of LVLMs: despite accounting for the majority of tokens in the input, image tokens receive significantly less attention. Conversely, system prompts, which provides the minimal semantic information, attract the most of the attention scores. To delve deeper into this phenomenon, we analyze the attention maps of the first, middle, and last layers during the decoding process of a model response as shown in Figure 4. The attention maps for all layers are provided in figure-7 of the supplement material.

From the attention visualization results, we can see that in shallow layer, the attention scores distribute more smoothly across different tokens. While in deep layer, there are vertical strong lines (in the system prompt) that takes up most of attention scores. The existence of vertical strong line shows that there are some input tokens that consistently received high attention during the whole decoding process. This also explains the highly imbalanced attention efficiencies in our statistics: A small portion of anchor tokens aggregate the information from all input tokens and the model much favors to attend to those anchor tokens in deep layers. Our findings also align with the information flow of Large Language Model found in Wang et al. (2023).

4 FastV

With insights from the validated phenomena and explanation, we propose FastV as a solution to reduce the inference costs of LVLMs without sacrificing the performance.

4.1 Dynamically Prune Vision Tokens

Figure 5 illustrates the general idea of FastV. The key is the image token re-rank and filtering module. It consists of one ranking function f_ϕ and two parameters: filtering layer K and filtering ratio $R\%$. At layer K of the LVLM, the ranking function f takes a sequence of input

tokens and rank them by certain importance criteria ϕ . The last $R\%$ tokens after ranking would be pruned out in successive layers. We simply compute the average attention-score one token received from all other tokens as the criteria ϕ_{attn} in our experiment. In extreme condition, K could be also set to 0, that image tokens are pruned before sending to the language model, we use random ranking as the criteria ϕ_{rand} where image tokens are randomly dropped.

FastV is plug-and-play to different token-based LVLMS for various vision language tasks without the need of training the model. We take video understanding tasks with VideoLLaVA Lin et al. (2023) as example as shown in Figure 5.

4.2 Computing Cost Estimation

We consider the computation of multi-head attention (MHA) and feed-forward network (FFN) module in the FLOPs estimation. For one transformer layer, assume n is the token number, d is the hidden state size, m is the intermediate size of FFN, the total FLOPs can be estimated by $4nd^2 + 2n^2d + 2ndm$. For the whole model, assume FastV prunes tokens from n to $\hat{n} = (1 - R\%) \cdot n$ after layer K and there are T layers at all. The theoretical FLOPs reduction ratio related to image tokens is computed as:

$$1 - \frac{K \times (4nd^2 + 2n^2d + 2ndm) + (T - K) \times (4\hat{n}d^2 + 2\hat{n}^2d + 2\hat{n}dm)}{T \times (4nd^2 + 2n^2d + 2ndm)} \quad (5)$$

We plot a 3D graph to show how the FLOPs reduction ratio changes with FastV’s parameter K and R in Figure 6.

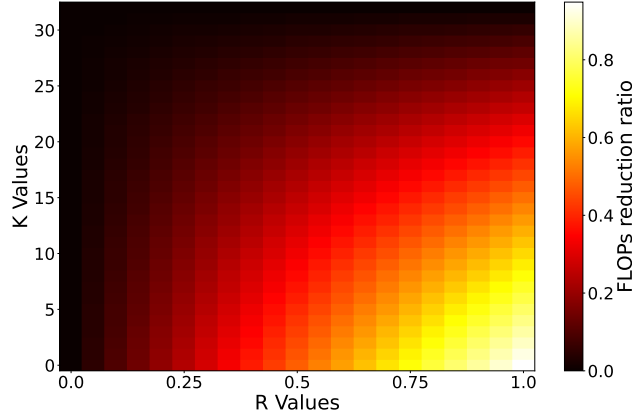


Figure 6: The heat map of theoretical FLOPs reduction ratio. The color in the figure represents the reduction ratio in different K and R in FastV.

4.3 Comparison: Training With Less Visual Tokens

FastV achieves computation reduction through eliminating redundant visual tokens during inference stage. An alternative method to reduce visual tokens is directly training with less visual tokens. This could be simply done by conducting pooling on the output of visual encoder during LVLMS’s training process. We compare FastV and this method in our ablation studies (sec. 5.4).

Table 1: Performance/Computation Balance of FastV under different configurations (K for filtering layer, R for filtering ratio). **Highest score** for each model is in red while the **second highest** is in blue.

Model	K	R	FastV Settings		Nocaps <i>CIDEr</i>	Flickr30k <i>CIDEr</i>	A-OKVQA <i>Accuracy</i>	MMMU <i>Accuracy</i>	Avg
			Flops(B)	Flops Ratio					
LLaVA-1.5-7B		Baseline	99.3	100%	99.8	67.9	76.7	34.8	69.8
	2	90%	19.9	20%	72.1	43.7	70.1	35	55.2
	2	75%	32.8	33%	94.6	63.6	75.5	34.8	67.1
	2	50%	54.6	55%	99.7	67.5	77	34.4	69.7
	3	90%	22.8	23%	87.2	55.8	71.9	34.8	62.4
	3	75%	34.8	35%	98	65	74.7	34.1	68.0
	3	50%	56.6	57%	99.7	68.3	76.7	34.3	69.8
	5	90%	27.8	28%	88.6	59.3	70.6	33.9	63.1
	5	75%	39.7	40%	98.5	66.3	74.8	34.3	68.5
	5	50%	59.6	60%	99.2	67.9	76.8	34.3	69.6
	0	90%	18.9	19%	7	53.2	66.8	34.7	40.4
	0	75%	28.8	29%	27.2	61.4	72.8	35.1	49.1
	0	50%	51.6	52%	100.9	65.5	75.3	34.3	69.0
LLaVA-1.5-13B		Baseline	154.6	100%	102.8	73	82	36.4	73.6
	2	90%	29.7	19%	87.9	62	75	36.3	65.3
	2	75%	50.2	32%	100.5	72.5	80.9	38.1	73.0
	2	50%	84.6	55%	103.1	73.4	81	36.7	73.6
	3	90%	33.0	21%	90.2	63.6	75.2	34.9	66.0
	3	75%	52.9	34%	100.9	72.1	79.5	36.4	72.2
	3	50%	86.4	56%	102.7	73.4	81.3	36.4	73.5
	5	90%	39.6	26%	93.5	67.4	75.8	35.4	68.0
	5	75%	58.4	38%	101.4	72.5	80	36.2	72.5
	5	50%	90.1	58%	102.5	73.5	81.2	36.6	73.5
QwenVL-Chat-7B		Baseline	71.9	100%	94.9	72.5	75.6	35.8	69.7
	2	90%	15.8	22%	81.9	61.5	68.5	35.3	61.7
	2	75%	24.4	34%	90.5	67.0	75.1	35.3	67.0
	2	50%	39.5	55%	94.4	71.4	75.3	35.6	69.2

Table 2: Experiments with more models and benchmarks.

Methods	AI2Diagram $\uparrow$	SciQA-IMG $\uparrow$	SeedBench $\uparrow$	MMVet $\uparrow$	MME $\uparrow$
LLaVA-1.5-13B	59.45	72.99	68.23	30.55	1827.75
+ FastV (K=2,R=50%)	58.96	73.23	68.03	31.25	1849.68
InstructBLIP-Vicuna-13B	45.46	61.15	52.11	24.19	1143.5
+ FastV (K=2,R=50%)	43.12	61.23	50.41	22.15	1129.8
+ FastV (K=5,R=50%)	44.39	62.33	51.69	23.51	1140.5

Table 3: Fine-grained results on MME benchmark.

Methods	Exist.	Count	Position	Color	OCR	Poster	Celeb.	Scene	Landmark	Art.	Comm.	Num.	Text.	Code.	Total
LLaVA-1.5-13B	185.00	155.00	133.33	170.00	125.00	160.72	152.54	161.25	170.50	118.50	128.41	42.50	77.50	47.50	1827.75
+ FastV (K=2,R=50%)	185.00	155.00	133.33	175.00	132.50	159.77	153.15	161.75	168.25	117.00	126.43	42.50	82.50	57.50	1849.68

5 Experiment

5.1 Evaluation Tasks

We conduct a wide range of evaluation including image captioning, VQA, multimodal reasoning, video QA and fine-grained benchmarks like MME Fu et al. (2023) to examine the influence of FastV on the performance of LVLMS. We use greedy search for all experiments and provide details for each task in section A in the supplement material.

5.2 Model Settings

We test FastV with various open source models. For image understanding tasks, we conduct experiments on LLaVA1.5-7B, 13B Liu et al. (2023b), and Qwen-VL Bai et al. (2023). When it comes to video understanding tasks, our baseline model is VideoLLaVA Lin et al. (2023). We adopt the settings as reported in their paper for the baseline models.

Table 4: Real inference budget comparison between FastV and vanilla decoding. To get rid of the influence of output sequence length on decoding time, we report the result on A-OKVQA dataset where the model only needs to output an option. With FastV, an 13B model could inference as fast as a 7B model while maintaining its superior performance. The latency experiments are conducted on single A40 GPU.

Model	Total-Time	GPU-Memory	Score	Latency/Example
LLaVA-1.5-7B	6:34	19G	76.7	0.344s
w/ FastV (K=0, R=50%)	4:23	16G	75.3	0.230s
LLaVA-1.5-13B	10:17	38G	82.0	0.539s
w/ FastV (K=0, R=50%)	6:30	30G	80.5	0.341s

Table 5: Finegrained Results on PCA-Bench and OCR-VQA. P, C, and A each denotes Perception, Cognition and Action score. G-PCA denotes Genuine PCA score where the model must make correct perception, cognition and action for one test example to gain 1 score. The scores are averaged among all three domains including Auto-Driving, Domestic Robot and Open-World Game.

Model	FLOPs	PCA-Bench Open Test				PCA-Bench Closed Test				OCR-VQA Rouge-L
		P	C	A	G-PCA	P	C	A	G-PCA	
LLaVA-1.5-7B	99.3B	0.493	0.353	0.433	0.263	0.513	0.387	0.450	0.277	0.51
LLaVA-1.5-13B	154.6B	0.530	0.460	0.503	0.333	0.563	0.550	0.573	0.353	0.55
w/ FastV (K=0, R=50%)	78.9B	0.490	0.395	0.443	0.292	0.519	0.450	0.512	0.283	0.49
w/ FastV (K=2, R=50%)	84.6B	0.533	0.423	0.513	0.340	0.581	0.545	0.580	0.368	0.55
w/ FastV (K=2, R=75%)	50.2B	0.513	0.417	0.483	0.320	0.523	0.510	0.533	0.323	0.54

5.3 Main Results

Image Understanding. The performance on tasks under different FastV settings are shown in Table 1 (Nocaps, Flickr30k, A-OKVQA, MMMU) and Table 5 (PCA-Bench, OCR-VQA). The result of latency test is shown in Table 4.

In Table 1, we present the performance trend with FLOPs ratio ranging from 19% to 100% by FastV, for different type and size of models. We also plot the relation between FLOPs Reduction ratio (1-FLOPs Ratio) and average performance in Figure 1. The results indicate that FastV (K=2, R=50%) could achieve about 45% FLOPs reduction for different LVLMS without sacrificing the performance. The FLOPs-Performance trade-off is also highly adjustable by lowering K and increasing R if we want to pursue an ultimate speed up. As shown in the latency test (Table 4), an 13B model with FastV could inference as fast as a 7B model with superior performance for A-OKVQA.

In PCA-Bench and OCR-VQA, (Table 5), which runs finegrained analysis on perception, cognition, action and OCR abilities, we find that FastV (K=2, R=50%) could maintain the sub-scores while significantly decreasing the FLOPs.

Video Understanding. The results of FastV on different video question answering tasks in shown in table 6 (TGIF, MSVD, MSRVT). To our surprise, we find FastV could generally improves the Video-QA tasks performance while saving 40%+ computations especially for the TGIF task. We think the main reason is that the redundancy information problem is more severe for video understanding as multiple images from the video are transformed to tokens when sending to the LLM. For example, an image costs 576 tokens in LLaVA1.5 model, while a video costs 2048 tokens in Video-LLaVA. As shown in the case from Figure 5, setting suitable FastV parameters could lead to much FLOPs reduction for Video-LLaVA while the outputs are nearly identical.

Fine-grained Benchmarks and More Models We conduct additional experiments with InstructBLIP and also with more fine-grained LVLMS benchmarks such as SciQA-IMGLu et al. (2022), SeedBench Li et al. (2023a), MMVet Yu et al. (2023), and MME Fu et al. (2023), together with benchmarks requiring more visual processing such as AI2Diagram. The results and fine-grained scores of MME are shown in Table 2 and Table 3. FastV works well

Table 6: GPT-Evaluation Results on Video Question Answering Tasks.

Model	TGIF		MSVD		MSRVTT		Avg	
	Acc	Score	Acc	Score	Acc	Score	Acc	Score
Video-LLaVA (Flops=100%)	0.18	2.5	0.70	3.9	0.56	3.5	0.48	3.3
w/ FastV (K=2, R=50%, Flops=52.3%)	0.21	2.6	0.71	3.9	0.55	3.5	0.49	3.3
w/ FastV (K=5, R=50%, Flops=57.1%)	0.20	2.6	0.71	4.0	0.57	3.5	0.49	3.4

Table 7: Ablation studies results. Scores labelled as “Failed” denotes the model could not follow instructions to generates valid results for evaluation.

Model	Nocaps	Flickr30k	A-OKVQA	MMMU
LLaVA1.5-7B (Retrained)	100.3	70.2	78.5	34.5
(a) w/ Train with 50% image tokens	98.5	68.5	76.8	33.5
(b) w/ FastV (K=2, R=50%)	100.1	70	78.4	34.6
(c) w/ FastV (K=2, R=50%, Random)	99.5	68.3	78.2	34.2
(d) w/ FastV (system prompt)	89.2	64.3	69.2	33.8
(e) w/ FastV (prune first half system prompt)	17.5	27.8	Failed	Failed
(f) w/ FastV (instruction)	77.3	50.1	56.5	29.5
(g) w/ StreamingLLM Xiao et al. (2023)	13.2	21.4	Failed	Failed

on different LVLM benchmarks with competitive performance. We find that InstructBLIP shows slightly more performance degradation than LLaVA with same FastV config. The gap soon closes when we just set K to 5. We think it’s because Q-Former initially reduces image tokens, resulting in direct information loss. Consequently, it requires adjusting the FastV parameters to avoid too much information loss.

5.4 Ablation Studies

Balance between Cost and Performance. We conduct an ablation experiment on how the parameters (K and R) influence the acceleration and downstream task’s performance. We select OCR-VQA as the task, which necessitates a through understanding of the image. The result is shown in Figure 7. **When K is small, lowering R would improve the performance with a smaller FLOPs reduction ratio.** In contrast, **when K is large, adjusting R has minimal impact on the overall performance.** This observation further proves that in deep layers, there is high redundancy in image tokens.

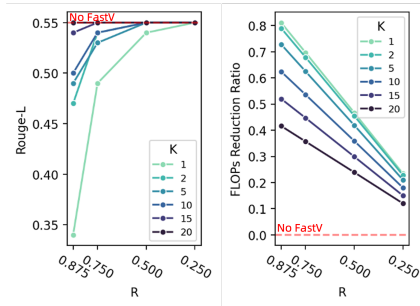


Figure 7: Ablation study on filtering layer K and filtering ratio R in FastV. Experiments are conducted with LLaVA1.5-13B on OCR-VQA task. When K is small, lowering R would improve the performance with a smaller FLOPs reduction ratio. In contrast, when K is large, changing R has minimal impact on the overall performance.

Training with Less Tokens. FastV reduces computational requirements (FLOPs) by pruning tokens during the inference stage. An alternative approach for token reduction involves training the LVLM at a lower resolution. To facilitate a fair comparison, we retrained two LLaVA1.5-7B models, adhering to the original pretraining and supervised finetuning proto-

cols. The sole modification in the second model’s training process was the incorporation of an average pooling layer (with a stride of 2) following the Clip encoder, leading to a 50% reduction in image tokens during training. A comparison between lines (a) and (b) in Table 7 reveals that reducing the input resolution directly during training results in diminished performance. Conversely, FastV manages to decrease the number of image tokens without compromising performance, showcasing its efficiency in balancing computational savings with model efficacy.

Pruning Token Strategy. FastV strategically reduces the number of image tokens during the inference phase of LVLMs, motivated by our observation that image tokens exhibit the lowest attention efficiency relative to other types of input tokens. In experiments detailed in lines (d) and (f) of the study, we specifically pruned tokens that were not related to images, such as system prompts and instruction tokens. This selective pruning resulted in significant performance declines, even when only a minimal number of non-image tokens were removed. We also compare randomly drop visual tokens instead of dropping by attention rank, as shown in line (c). It resulted in declined results compared with origin FastV (b). These findings underscore the distinct roles that visual and textual tokens play within LVLMs. It highlights FastV’s effectiveness in precisely targeting image tokens for reduction, thereby optimizing performance without compromising the model’s overall functionality.

In our previous observation about attention efficiency, we find out that the system prompt takes up of most attention even if they carry the least semantic information in the context. We conduct another experiment by directly prune the first half tokens of the system prompt. Comparing line (d) and (e), we can find that the head tokens in the system prompt have dominant effect on the model performance. Our findings also align with StreamingLLM Xiao et al. (2023) where they find that the first 4 tokens in LLM play the most important role during inference. However, directly applying the same sparse attention pattern as StreamingLLM would lead to a substantial degradation in LVLM’s performance as shown in line (g) of Table 7. This suggests a fundamental difference in how image tokens, as opposed to text tokens, contribute to the information processing within LLMs.

6 Conclusion

In this paper, we propose FastV, a plug-and-play inference cost optimization method for Large Vision-Language Models. Our insight for FastV arises from our observation that the attention computation over visual tokens is of extreme inefficiency in the deep layers of popular LVLMs though they take up a large portion of input tokens. FastV prunes out the unnecessary visual tokens according to the attention score ranking, which results in significant inference cost reduction without sacrificing performance.

References

- Harsh Agrawal, Peter Anderson, Karan Desai, Yufei Wang, Xinlei Chen, Rishabh Jain, Mark Johnson, Dhruv Batra, Devi Parikh, and Stefan Lee. nocaps: novel object captioning at scale. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pp. 8947–8956, 2019.
- Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A frontier large vision-language model with versatile abilities. *ArXiv preprint*, abs/2308.12966, 2023.
- Rohan Bavishi, Erich Elsen, Curtis Hawthorne, Maxwell Nye, Augustus Odena, Arushi Somani, and Sağnak Taşlılar. Introducing our multimodal models, 2023. URL <https://www.adept.ai/blog/fuyu-8b>.
- Qingqing Cao, Bhargavi Paranjape, and Hannaneh Hajishirzi. Pumer: Pruning and merging tokens for efficient vision language models, 2023. URL <https://arxiv.org/abs/2305.17530>.
- Liang Chen, Yichi Zhang, Shuhuai Ren, Haozhe Zhao, Zefan Cai, Yuchi Wang, Peiyi Wang, Tianyu Liu, and Baobao Chang. Towards end-to-end embodied decision making via multi-modal large language model: Explorations with gpt4-vision and beyond. *ArXiv*, 2023.
- Liang Chen, Yichi Zhang, Shuhuai Ren, Haozhe Zhao, Zefan Cai, Yuchi Wang, Peiyi Wang, Xiangdi Meng, Tianyu Liu, and Baobao Chang. Pca-bench: Evaluating multimodal large language models in perception-cognition-action chain. 2024.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022.
- Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model. volume abs/2303.03378, 2023.
- Chaoyou Fu, Peixian Chen, Yunhang Shen, Yulei Qin, Mengdan Zhang, Xu Lin, Zhenyu Qiu, Wei Lin, Jinrui Yang, Xiawu Zheng, et al. Mme: A comprehensive evaluation benchmark for multimodal large language models. *arXiv preprint arXiv:2306.13394*, 2023.
- Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms, 2024.
- Yunseok Jang, Yale Song, Youngjae Yu, Youngjin Kim, and Gunhee Kim. Tgif-qa: Toward spatio-temporal reasoning in visual question answering, 2017.
- Dan Kondratyuk, Lijun Yu, Xiuye Gu, José Lezama, Jonathan Huang, Rachel Hornung, Hartwig Adam, Hassan Akbari, Yair Alon, Vighnesh Birodkar, Yong Cheng, Ming-Chang Chiu, Josh Dillon, Irfan Essa, Agrim Gupta, Meera Hahn, Anja Hauth, David Hendon, Alonso Martinez, David Minnen, David Ross, Grant Schindler, Mikhail Sirotenko, Kihyuk Sohn, Krishna Somandepalli, Huisheng Wang, Jimmy Yan, Ming-Hsuan Yang, Xuan Yang, Bryan Seybold, and Lu Jiang. Videopoet: A large language model for zero-shot video generation, 2023.
- Zhenglun Kong, Peiyan Dong, Xiaolong Ma, Xin Meng, Mengshu Sun, Wei Niu, Xuan Shen, Geng Yuan, Bin Ren, Minghai Qin, Hao Tang, and Yanzhi Wang. Spvit: Enabling faster vision transformers via soft token pruning, 2022. URL <https://arxiv.org/abs/2112.13890>.

- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023.
- Bohao Li, Rui Wang, Guangzhi Wang, Yuying Ge, Yixiao Ge, and Ying Shan. Seed-bench: Benchmarking multimodal llms with generative comprehension, 2023a. URL <https://arxiv.org/abs/2307.16125>.
- Juncheng Li, Kaihang Pan, Zhiqi Ge, Minghe Gao, Hanwang Zhang, Wei Ji, Wenqiao Zhang, Tat-Seng Chua, Siliang Tang, and Yueting Zhuang. Empowering vision-language models to follow interleaved vision-language instructions. *arXiv preprint arXiv:2308.04152*, 2023b.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. *ArXiv preprint, abs/2301.12597*, 2023c.
- Yanwei Li, Chengyao Wang, and Jiaya Jia. Llama-vid: An image is worth 2 tokens in large language models, 2023d.
- Zhang Li, Biao Yang, Qiang Liu, Zhiyin Ma, Shuo Zhang, Jingxu Yang, Yabo Sun, Yuliang Liu, and Xiang Bai. Monkey: Image resolution and text label are important things for large multi-modal models. *arXiv preprint arXiv:2311.06607*, 2023e.
- Youwei Liang, Chongjian Ge, Zhan Tong, Yibing Song, Jue Wang, and Pengtao Xie. Not all patches are what you need: Expediting vision transformers via token reorganizations, 2022. URL <https://arxiv.org/abs/2202.07800>.
- Bin Lin, Bin Zhu, Yang Ye, Munan Ning, Peng Jin, and Li Yuan. Video-llava: Learning united visual representation by alignment before projection. *arXiv preprint arXiv:2311.10122*, 2023.
- Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context, 2023a.
- Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with ringattention, 2024a.
- Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning, 2023b.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *ArXiv preprint, abs/2304.08485*, 2023c.
- Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024b. URL <https://llava-vl.github.io/blog/2024-01-30-llava-next/>.
- Jiasen Lu, Christopher Clark, Sangho Lee, Zichen Zhang, Savya Khosla, Ryan Marten, Derek Hoiem, and Aniruddha Kembhavi. Unified-io 2: Scaling autoregressive multimodal models with vision, language, audio, and action, 2023.
- Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 2507–2521. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/11332b6b6cf4485b84afadb1352d3a9a-Paper-Conference.pdf.
- Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. Video-chatgpt: Towards detailed video understanding via large vision and language models. *arXiv:2306.05424*, 2023.

- Anand Mishra, Shashank Shekhar, Ajeet Kumar Singh, and Anirban Chakraborty. Ocr-vqa: Visual question answering by reading text in images. In *2019 international conference on document analysis and recognition (ICDAR)*, pp. 947–952. IEEE, 2019.
- OpenAI. Gpt-4v(ision) system card. 2023.
- Bryan A Plummer, Liwei Wang, Chris M Cervantes, Juan C Caicedo, Julia Hockenmaier, and Svetlana Lazebnik. Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models. In *Proceedings of the IEEE international conference on computer vision*, pp. 2641–2649, 2015.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pp. 8748–8763, 2021.
- Dustin Schwenk, Apoorv Khandelwal, Christopher Clark, Kenneth Marino, and Roozbeh Mottaghi. A-okvqa: A benchmark for visual question answering using world knowledge. In *Computer Vision—ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part VIII*, pp. 146–162. Springer, 2022.
- Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA*, pp. 5998–6008, 2017.
- Ramakrishna Vedantam, C. Lawrence Zitnick, and Devi Parikh. Cider: Consensus-based image description evaluation, 2015.
- Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception, 2024.
- Lean Wang, Lei Li, Damai Dai, Deli Chen, Hao Zhou, Fandong Meng, Jie Zhou, and Xu Sun. Label words are anchors: An information flow perspective for understanding in-context learning. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9840–9855, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.609. URL <https://aclanthology.org/2023.emnlp-main.609>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv*, 2023.
- Yizhe Xiong, Hui Chen, Tianxiang Hao, Zijia Lin, Jungong Han, Yuesong Zhang, Guoxin Wang, Yongjun Bao, and Guiguang Ding. Pyra: Parallel yielding re-activation for training-inference efficient task adaptation, 2024. URL <https://arxiv.org/abs/2403.09192>.
- Dejing Xu, Zhou Zhao, Jun Xiao, Fei Wu, Hanwang Zhang, Xiangnan He, and Yueting Zhuang. Video question answering via gradually refined attention over appearance and motion. In *Proceedings of the 2017 ACM on Multimedia Conference, MM 2017, Mountain View, CA, USA, October 23–27, 2017*, pp. 1645–1653, 2017a.
- Dejing Xu, Zhou Zhao, Jun Xiao, Fei Wu, Hanwang Zhang, Xiangnan He, and Yueting Zhuang. Video question answering via gradually refined attention over appearance and motion. In *ACM Multimedia*, 2017b.

Weihao Yu, Zhengyuan Yang, Linjie Li, Jianfeng Wang, Kevin Lin, Zicheng Liu, Xinchao Wang, and Lijuan Wang. Mm-vet: Evaluating large multimodal models for integrated capabilities, 2023. URL <https://arxiv.org/abs/2308.02490>.

Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhui Chen. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *arXiv preprint arXiv:2311.16502*, 2023.

Haozhe Zhao, Zefan Cai, Shuzheng Si, Xiaojian Ma, Kaikai An, Liang Chen, Zixuan Liu, Sheng Wang, Wenjuan Han, and Baobao Chang. Mmicl: Empowering vision-language model with multi-modal in-context learning. *ArXiv preprint*, abs/2309.07915, 2023.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded, 2024.

Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *ArXiv preprint*, abs/2304.10592, 2023.

A Appendix

B Evaluation Tasks Description

Image Captioning. Image captioning requires the model to generate a description for a given image. We choose Nocaps Agrawal et al. (2019) and Flickr30k Plummer et al. (2015) as benchmarks and report CIDEr score Vedantam et al. (2015) as metric. For image captioning tasks Nocaps and Flickr30k, we adopt prompt as “Describe the image in one sentence.”

Visual Question Answering (VQA). VQA requires the model to generate an answer for a given image-question pair. We select the development set of A-OKVQA Schwenk et al. (2022) and the test set of OCR-VQA Mishra et al. (2019) as the benchmark and the report the multiple choice (MC) score of AOKVQA and Rouge-L score of OCR-VQA. For AOKVQA, we adopt the the multiple choice version of evaluation and use prompt as: “Analyse the image and choose the best answer for the following question: {question} Options: {options}. Output the letter of the correct answer.” For OCRVQA, we use the default question as prompt for each example as provided in the official dataset.

Multimodal Reasoning. Compared with VQA, multimodal reasoning requires more advanced perception, knowledge and reasoning skills of the model, which are more suitable benchmarks to evaluate the integrated abilities of LVLMS. We choose MMMU and PCA-Bench Chen et al. (2024) as benchmarks. MMMU is a multimodal benchmark featuring multi-discipline tasks demanding college-level subject knowledge and reasoning skills. PCA-Bench is a complex embodied reasoning benchmark with error localization, which features three different domains including autonomous driving, robot and game. We report the multiple choice accuracy for the development set of MMMU and Perception, Cognition, Action, Genuine PCA scores for both the open and closed test set of PCA-Bench. We use the default prompts for each example as provided in the official dataset MMMU and PCA-Bench.

Video Question Answering. Similar to VQA for single image, Video Question Answering requires the model to generate answer given a video-question pair. Current LVLMS usually deal with video question answering tasks by sampling multiple frames as input, resulting in longer image token sequences. We choose TGIF-QA Jang et al. (2017), MSVD-QA Xu et al. (2017b) and MSRVT-QA Xu et al. (2017a) as benchmarks following the evaluation pipeline of Video-ChatGPT Maaz et al. (2023) and report the accuracy and chatgpt-score as metrics. We use the first 1K examples in each benchmark in our experiments due to the limited

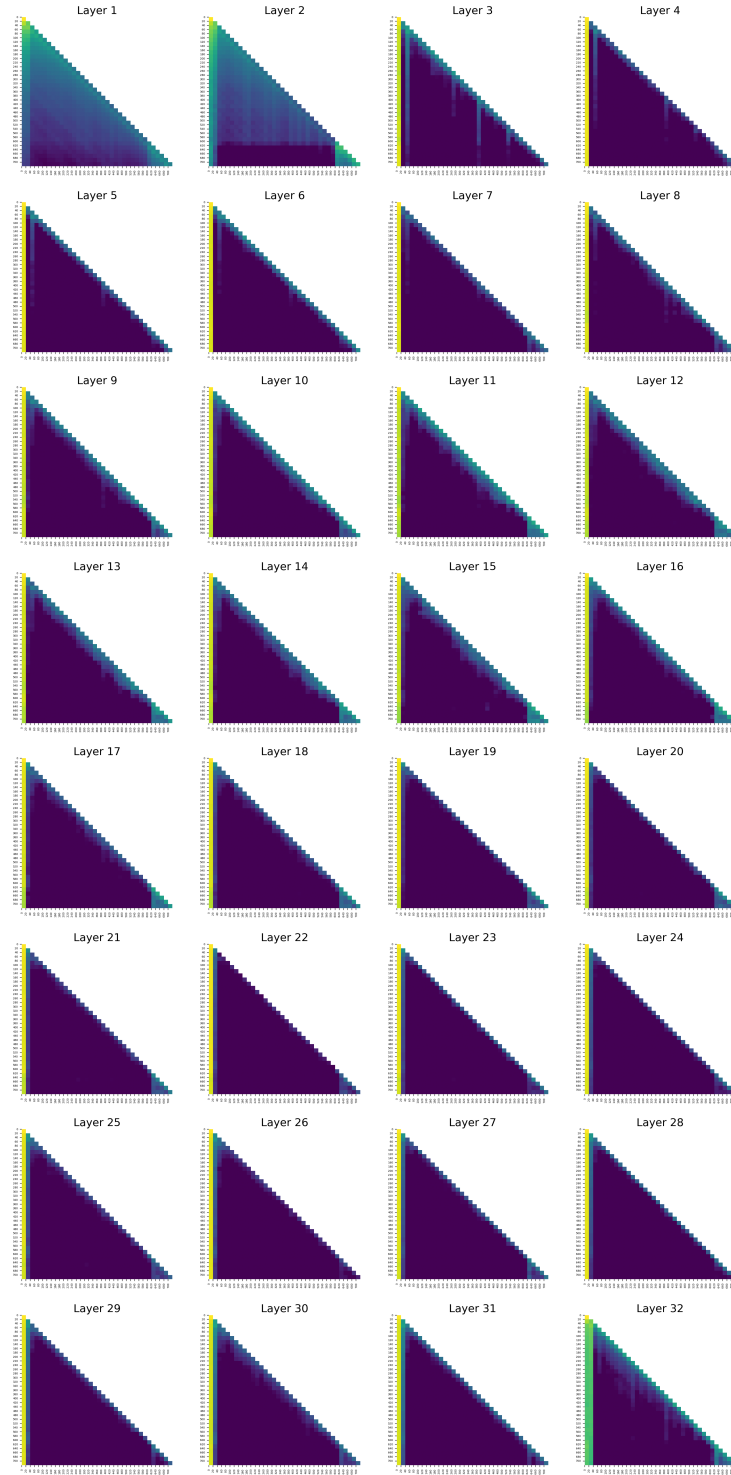


Figure 8: Full Attention Maps of Each Layer of LLaVA.

commercial API usage in evaluation. For all video QA tasks, we use the default question as the prompt as provided in Video-LLaVA, and use the same tool from Video-ChatGPT to conduct GPT evaluation.

Fine-grained Benchmarks For the evaluation of the influence of FastV on LVLM performance, we incorporate four distinct Fine-grained benchmarks: MME Fu et al. (2023), Seed-Bench Li et al. (2023a), SciQA-IMG Lu et al. (2022), and MMVet Yu et al. (2023). MME offers a comprehensive evaluation of models’ perception and cognition abilities across a diverse set of tasks, focusing on intuitive and quantifiable analysis without extensive prompt engineering. SEED-Bench, on the other hand, evaluates generative comprehension across multiple dimensions, ensuring question relevance and quality through a mix of automated filtering and manual verification. While MME and SEED-Bench cover general abilities of LVLMs, SciQA-IMG and MMVet focus on the advanced aspects of multi-modal understanding. SciQA-IMG is a large-scale multimodal science question dataset annotated with detailed lectures and explanations. MMVet evaluates LVLMs on complex multimodal tasks, emphasizing multi-modal understanding and free-form answering capabilities, thus offering a comprehensive view of model performance.